

DES Multi-Block Encryption/Decryption System-on-Chip Tape-In

Group 02

Amber Li (adl94), Yunnice Kim (yk536), Kyle Mui (km776)

1. Introduction

Our Project 3 idea was to design a DES encryption and decryption accelerator that can process multi-block messages. Cryptography is essential for privacy, security, and everyday operation in today's world, as data is constantly stored and shared online. We wanted to explore the acceleration potential of cryptographic algorithms in software versus hardware [1]. In addition, we compared two encryption modes: Electronic Codebook (ECB) mode, which encrypts one 64-bit block at a time, and Counter (CTR) mode, a multi-block cipher mode. The accelerator can encrypt/decrypt longer messages by chunking them into 64 bit blocks. The motivation for the ECB vs CTR comparison comes from the added security benefits of CTR. Both modes involve instantiating multiple DES core modules, but ECB mode operates directly on the plaintext, and therefore is prone to revealing patterns [2], while CTR performs the DES operations on a nonce counter.

Accelerators for use of cryptography algorithms have been utilized in industry and extensively studied in academia. Work by Rouvroy et al. demonstrated that an FPGA-based DES accelerator can significantly outperform software by exploiting parallelism and pipelining, achieving high throughput through replicating encryption cores [3]. Their work emphasizes the advantage of hardware to parallelize the independent data blocks, making it well suited for modes like CTR. The study showed that increasing parallelism yields substantial performance gains at the cost of increased area usage. Satoh et al. proposed a compact, high-speed AES architecture by optimizing the substitution boxes (S-boxes) as well as round and key schedule logic [4]. Although the work targets AES, the design principles can be transferable to DES, which contains similar logic structures. They employ techniques such as reducing redundant logic and carefully structuring round operations to achieve high speeds. Another study on AES investigates the performance of ECB and CTR modes on FPGA-based hardware acceleration [5]. The paper demonstrates that these modes are especially well suited for hardware acceleration because the block level independence leads to substantial spatial parallelism via parallel datapaths. They achieved throughput improvements of up to 7x over CPU-based implementations. However, they also found that memory transfer latency and communication overhead become increasingly important for smaller message sizes.

Looking at industry applications, Rambus developed their 3DES-IP-16 accelerator, which supports single DES and triple DES in ECB, CBC, CFB and OFB modes up to 4 Gbps at 800MHz [6]. A paper published in ACM Transactions on Reconfigurable Technology and Systems (TRETs) explores a solution combining CBC and CTR modes of operation with a hardware context switching, tackling the problem of accelerating multiple coexisting streams by allowing the accelerator to treat the data as a singular stream [7]. In the paper, they were able to achieve a low performance overhead and a performance comparable to that of existing hardware accelerators that do not handle multiple streams.

We received help from Professor Batten during our project meetings and from the examples provided by the course staff. These provided a reference framework for structuring our design and helped us understand how to use new accelerator and memory architectures that we were not familiar with.

2. Baseline Design

The baseline design implemented a software version of the DES engine to explore the different versions of DES algorithms and provide a baseline for comparison against the accelerator implementation. Our software baseline was written in C and runs on the TinyRV2 processor. The TinyRV2 processor is a 5-stage, pipelined processor that supports full bypassing. It uses a single-cycle integer multiplier and can handle CSRs for interacting with medium-grain accelerators [8]. The cache design for the instruction and data caches is a 2-way set-associative cache using a write-back write-allocate miss policy. The cache's datapath and control unit were adapted to use synchronous-read SRAMs [9], allowing for a read operation to return data in the following cycle. The processor uses

a register-sized interface, with 4-byte requests/responses. The pmx-sim was used to cross-compile our C baseline and run it on the ISA simulator.

Our baseline design built off of our Project 2 DES Encryption Accelerator software baseline. The general DES encryption algorithm [10] takes in a 64-bit plaintext and key and produces a 64-bit ciphertext through two main operational paths: a key path and a data path. The key path uses the 64-bit key to generate 16 subkeys through a PC-1 permutation, left circular shifts, and a PC-2 permutation. The data path first applies an initial permutation (IP) onto the 64-bit plaintext, then splits the result into 32-bit left and right halves to go through 16 Feistel rounds. Each Feistel round consists of an Expansion (E) permutation, an XOR with the round's subkey, 8 S-box substitutions, and a P permutation. The right half of the current round becomes the left half for the next round, while the left half of the current round is XORed with the Feistel round output to become the right half for the next round. After 16 rounds, the halves undergo a final permutation (FP) to generate the 64-bit output ciphertext, as shown in Figure 1 below.

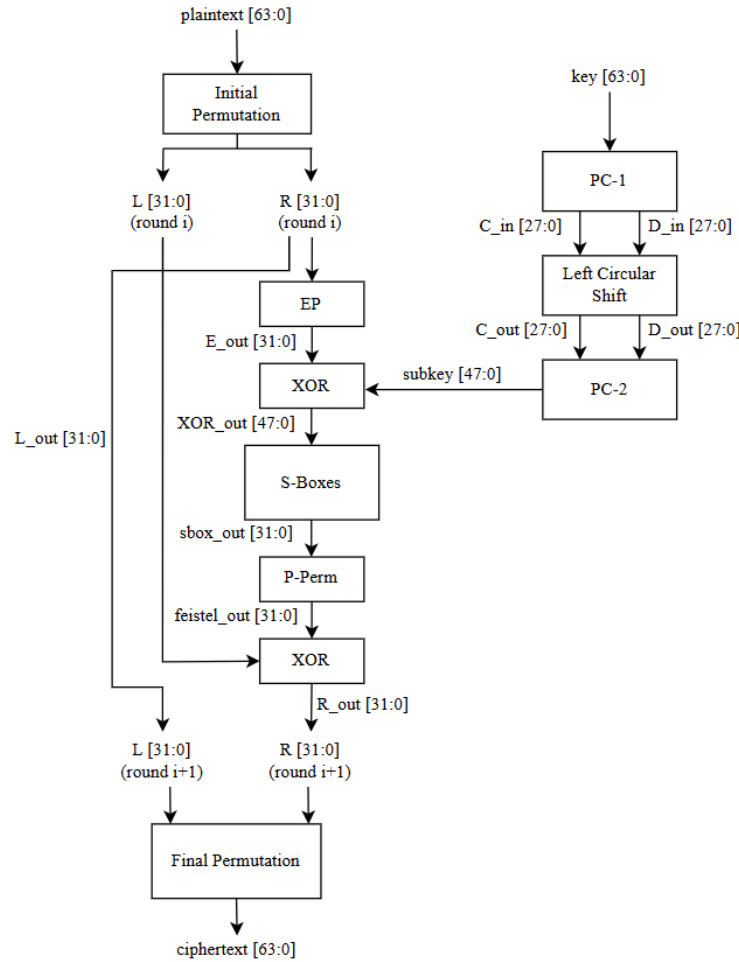


Figure 1. DES Algorithm Block Diagram

Our software baseline used certain optimizations to improve its performance for a better comparison to our alternative accelerator design. Because DES was designed for hardware with many bit-level operations, the algorithm became extremely inefficient on general-purpose processors that work best with bytes or words and must run a series of instructions to execute the same operation. The first major optimization was combining the S-box and P permutations [11] into one SP-box, which was generated using a Python script. Similarly, the E-permutation and the XOR were fused together, removing the intermediate storage and shifting/masking. The IP and FP operations were configured using a bitwise permutation helper perm_op that performs multiple bit swaps at once. The elimination of permutation steps was crucial because each bit shuffle added to the instruction count, and the

permutations were over 32 bits each and across 16 rounds. Using lookup tables instead led to high memory latency cost. In addition, the Feistel round loop was unrolled to remove loop overhead, and most functions were inlined for compiler optimizations. This optimized baseline design provides a better comparison to the hardware accelerator since it eliminates as many cost hits as possible as a result of the DES algorithm's unsuitability in software.

For Project 3, we added in decryption functionality as well as CTR mode, which allows processing of multiple 64-bit blocks. In ECB mode, the decryption function uses the same structure as encryption, but applies the subkeys to each Feistel round in reverse order (Figure 2). The symmetric property of the DES algorithm allowed for the same datapath to be reused for both operations with minimal modification. Since the key schedule generates the same 16 subkeys regardless of direction, the main change was reversing the order in which they are applied across the 16 Feistel rounds.

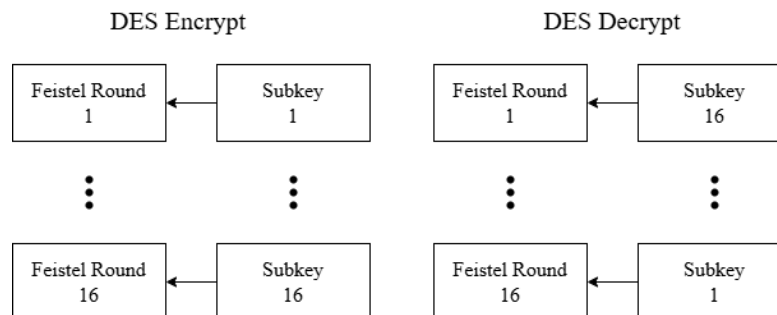


Figure 2. DES ECB Encryption and Decryption

CTR mode was added to enable encryption and decryption of messages spanning multiple 64-bit blocks. This was achieved by generating a keystream, where each block has a counter value that was encrypted with the key, and then XORed with the plaintext to produce the ciphertext [12]. The counter value was generated by adding the input nonce value to the current block number, which increments by one per successive block allowing for each block to produce a unique keystream independent of the others. Our software splits all 64-bit values into a high and a low half, so we manually implemented this addition. The block number was added to the low word, then the carry value was stored based on overflow detection logic and propagated to the high word. The same DES encrypt function was used except this counter value was encrypted with the key rather than directly encrypting the plaintext. The plaintext was incorporated via an XOR to yield the ciphertext. This means that encryption and decryption are identical if the nonce, key, and block numbers are the same (Figure 3), allowing for CTR mode to be added without additional DES logic.

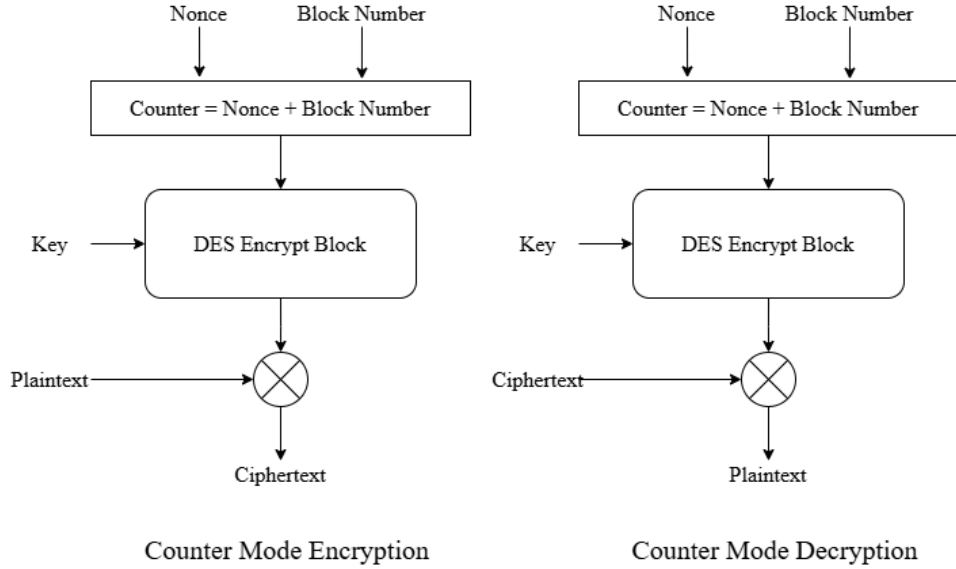


Figure 3. Counter Mode Encryption & Decryption Block Diagrams

3. Alternative Design

The alternative design is a hardware accelerator implementation of the cryptography engine. A wide accelerator was used with 128-bit message lengths, allowing the transmission of two DES messages at a time. This provides better bandwidth and leverages the parallelism of the chosen cryptography algorithms. The accelerator has direct access to memory via a data memory crossbar and generates memory read/write requests to retrieve the data without having to pass through the processor. The crossbar facilitates request arbitration, so that processor and accelerator requests do not clash and are handled fairly, as well as response routing, so the data goes back to the appropriate co-processor. Queues are used for the accelerator and memory request and response streams to account for crossbar delays and memory latency. The design follows that of a medium-grained accelerator, such that the processor configures the accelerator with initial values (source address, destination address, key, nonce, size) and prepares the data in memory, while the accelerator retrieves data from the memory, computes the result, and writes back into memory. With this setup, the CPU does not have to stream plaintext and ciphertext between it and the accelerator, avoiding lots of overhead.

The RTL design followed a modular approach, where each step within the DES algorithm was broken down into submodules for clarity and organization. Submodules include the S-boxes, subkey generation, Feistel round, and an overall DES round. These submodules were composed into a DES core module, with an FSM that schedules 16 rounds (Figure 4). These cores were instantiated in the accelerator designs, with each core able to encrypt and decrypt one 64-bit block. Two initial accelerators were designed, one using the ECB algorithm and one using CTR mode, both supporting encryption and decryption of two 64-bit messages at a time.

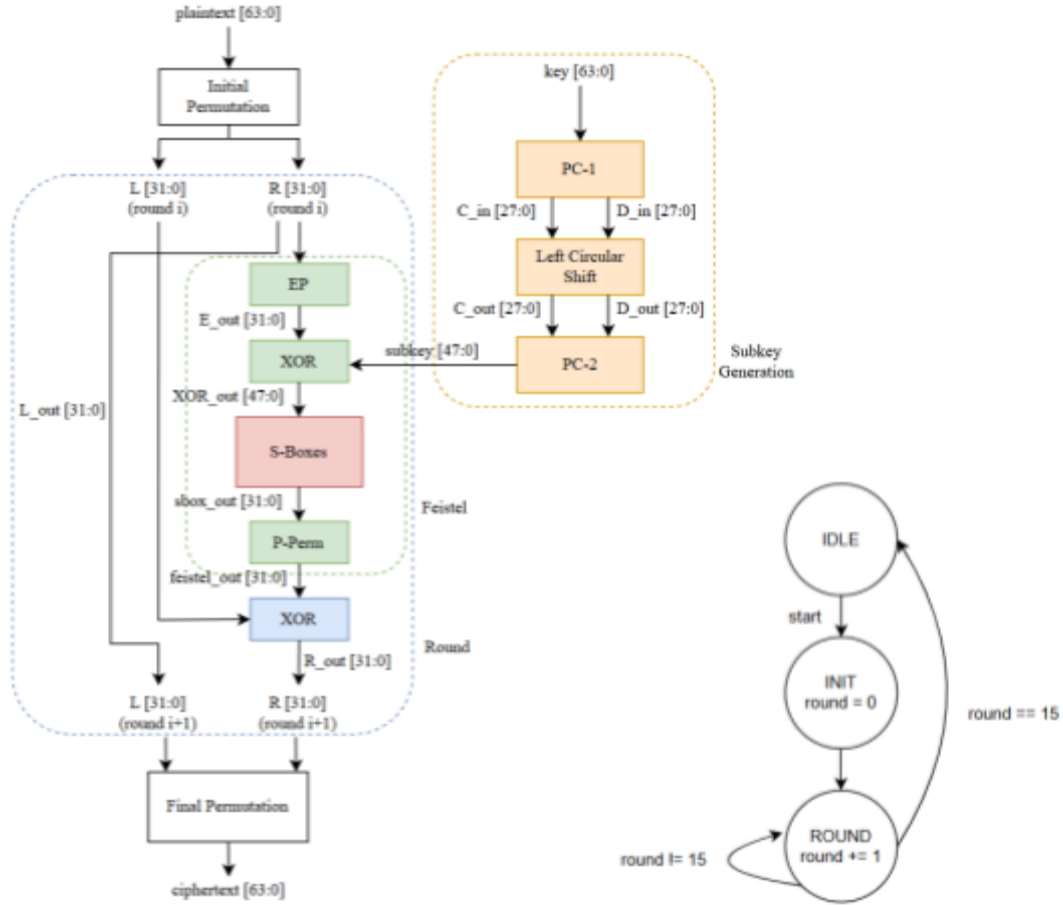


Figure 4. DES Core Submodules and FSM

3.1 ECB Version 1 Accelerator

The ECB accelerator uses DES cores that support both encryption and decryption. Decryption involves modifying the subkey generation module to generate all 16 subkeys at once, in order to apply them to each Feistel round in reverse. For version 1, the accelerator contains two DES cores to encrypt/decrypt two 64-bit blocks in parallel, for each memory message received. The accelerator can support processing of longer messages, as well as messages with an odd number of blocks. It uses a `second_block_valid` status signal to check if the second block at the given index is within bounds. Each index corresponds to one 128-bit memory pair of two blocks.

```
assign second_block_valid = ((idx << 1) + 32'd1) < num_blocks_reg;
```

The FSM contains 5 states: XCFG, M_RD, DES, M_WR, and WAIT (Figure 6).

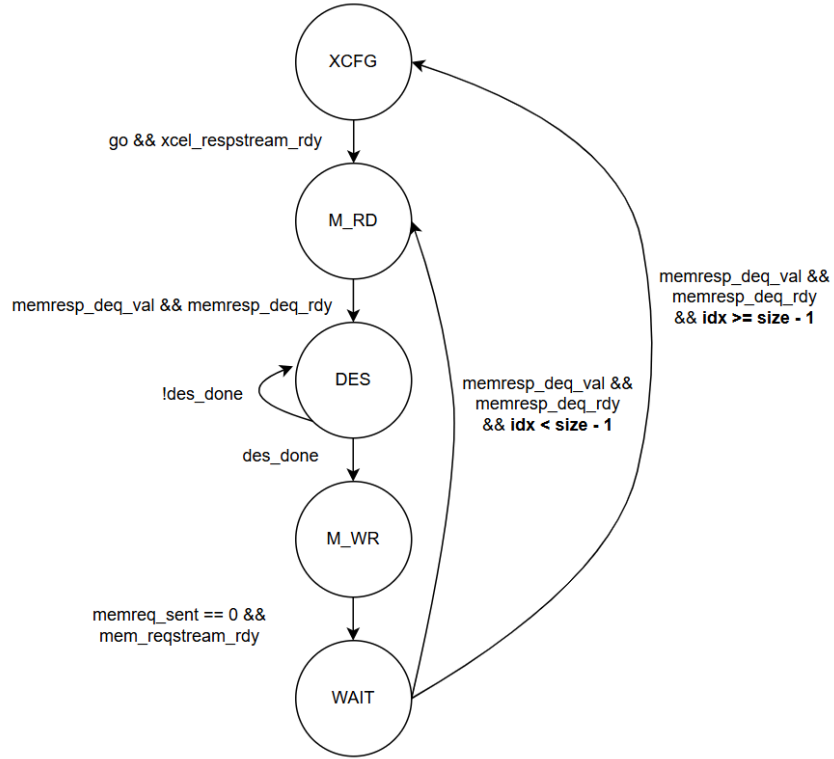


Figure 6. ECB Mode FSM

In STATE_XCFG, the accelerator begins accepting messages from the processor. The processor's message length is 4 bytes, so 64-bit data must be split into two halves. For a read request, the accelerator responds with a done value, signifying whether the accelerator is done processing all messages. For a write request, the processor sets the base address for where in memory to start reading, where in memory to write results to, the key, the decrypt mode, and the number of blocks (Table 1). It ties the accelerator processor handshake so that it only consumes a request if it can send its response. It also sends the go signal to trigger execution of the encryption/decryption and sets the index for block counting to 0. When go is set, and the request is fully consumed, the state transitions to STATE_M_RD.

Direction	Type	Register
Proc → Xcel	WRITE	go
Proc → Xcel	WRITE	base_src
Proc → Xcel	WRITE	base_dest
Proc → Xcel	WRITE	key[31:0]
Proc → Xcel	WRITE	key[63:32]
Proc → Xcel	WRITE	decrypt
Proc → Xcel	WRITE	size
Proc → Xcel	READ	done

Table 1. Proc Xcel Interface Messages

In the memory read state, the accelerator generates a memory read request using the base source address and the index. Since the memory messages are 128 bits wide, the index is shifted by 4 to accommodate 4 words at a time. The state uses control signals to keep track of the memory handshake. It sets `memreq_sent` when a memory request has been created and sent, which prevents sending duplicate requests. When the memory response is ready and valid, the accelerator receives the 128-bit response data and splits it into two 64-bit registers so they can be processed as two DES blocks. It also sets `memrsp_rcv` here, so the accelerator does not consume responses multiple times. The state also transitions to `STATE_DES` once the data successfully arrives.

In `STATE_DES`, the DES cores are run. A `des_start` signal is pulsed once when entering `STATE_DES` to start running both cores. The DES cores will run through the 16 Feistel rounds and produce the encrypted or decrypted output. Once `des_done` is high, which is set when both cores are done computing, the memory send and rcv signals are reset for the next memory transaction. In addition, the `second_block_valid` status signal is checked to assign block two if valid, and preserve what was in memory if not. Then the FSM transitions to `STATE_M_WR`.

The memory write state constructs a memory write request using the destination base address. It combines the resulting 64-bit text blocks into one 128-bit message and stores it in the appropriate spot in memory. Using the `second_block_valid` signal, it either writes both blocks using a full 128-bit message or writes only the first block with a 64-bit message. It then sets the `memreq_sent` signal high to indicate a message is being written. After `memreq_sent` returns to 0 and the memory request stream is ready, the FSM transitions to `STATE_WAIT`.

idx	Memory address	DES core 1	DES core 2
0	Base_src + 0x00	Block 0	Block 1
1	Base_src + 0x10	Block 2	Block 3
2	Base_src + 0x20	Block 4	Block 5

Table 2. Per-iteration Block Mapping

In `STATE_WAIT`, the accelerator waits for a write acknowledgement. Once memory confirms that the write was completed, the memory send and rcv signals are reset for the next memory message, and the index is checked to see whether there are still more blocks to process, based on the message size. If the index has not reached the message size, then it is incremented, and the state goes back to `STATE_M_RD` to repeat the DES CTR process. If it has reached the end, then the done bit is set high, `idx` is reset to 0, and the state goes back to `STATE_XCFG`. Table 2 shows an example of the block mapping for a message size of up to 6.

3.2 CTR Version 1 Accelerator

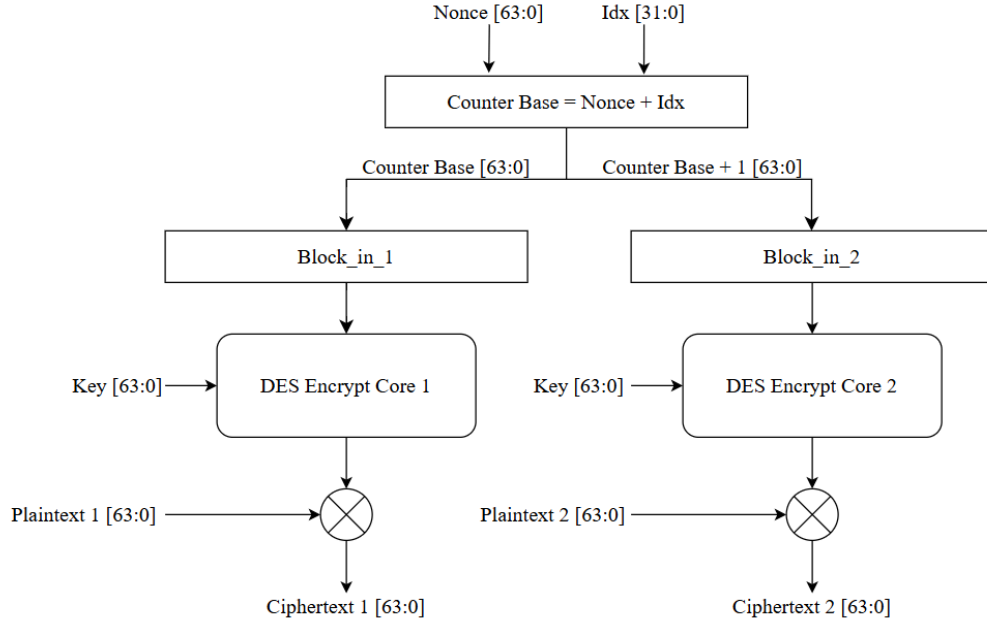


Figure 7. CTR Mode RTL Block Diagram

The CTR design uses encryption DES cores, plus some additional logic for tracking the counter and XORing the input text with the core output (Figure 7). Data registers are used to hold the plaintext, ciphertext, key, and nonce values used in the cryptography algorithm. These are set by the processor in the configuration state. To calculate the counter value for each block, the accelerator keeps track of the iteration index. This index counts how many 128-bit chunks, or 2 DES blocks, have been processed. To produce the counter base, the nonce value is added to the current index * 2, since we process 2 blocks at a time. Then this value is used as the DES encryption input for core 1, while counter base + 1 is used as the input for core 2. This computation scheme ensures that the correct counter value is calculated for each block solely from the index or current iteration number.

```
assign counter_base = nonce_reg + ({32'b0, idx} << 1);
assign block_in_1   = counter_base;
assign block_in_2   = counter_base + 64'd1;
```

Similar to ECB mode, CTR mode has no data dependency between blocks, since each block's output depends on the same nonce and key, and the only change across them is the block number and plaintext, which are known. This allows for parallelism in the DES cores. In addition, CTR mode is perfectly symmetric, so encryption and decryption can be run using the same hardware but feeding ciphertext in as the input, to derive the original plaintext, as long as the key and nonce are the same. This eliminates the need for an encrypt/decrypt control bit.

The CTR FSM is similar to the DES FSM with the same state transitions. The main differences are the counter logic, the input to the DES core, and the additional XOR in STATE_DES. As shown in the CTR mode diagram, the block being encrypted is the nonce + counter value. The output from the DES core is then XORed with the plaintext to produce the resulting ciphertext.

3.3 ECB and CTR Version 2 Accelerators

Version 2 of the accelerators restructures the RTL to scale from a fixed two-core design to a parametrized structure that takes advantage of parallelism (Figure 8). The NCORES parameter was added to the top modules, which is used throughout the design to support parallel processing of a variable number of DES cores. To instantiate multiple cores, a generate loop was used. All per-core input and output signals were extended to be arrays of size [NCORES-1:0], and the des_done and signals were vectorized and combined into one all_des_done control signal. These design choices allow each core to process its own block independently without the need for manually duplicating logic. Another key modification was the memory response queue depth, which is now parametrized to

match the number of messages required to feed all cores in a batch. This ensures memory responses can be buffered to fully populate the cores before computation begins, to maintain throughput.

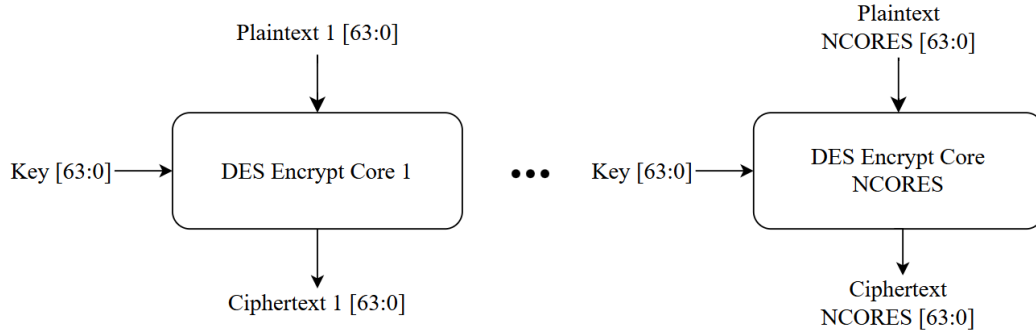


Figure 8. Parametrized DES Cores

The FSM logic in the memory states was updated to support batching. Instead of issuing a single memory request per iteration, the RTL calculates how many requests are needed to fill all cores. The M_RD and M_WR states loop to issue multiple read and write requests until memreq_sent reaches this value. Similarly, these states wait until all corresponding responses have been received by incrementing memrsp_rcv before transitioning out of the memory states.

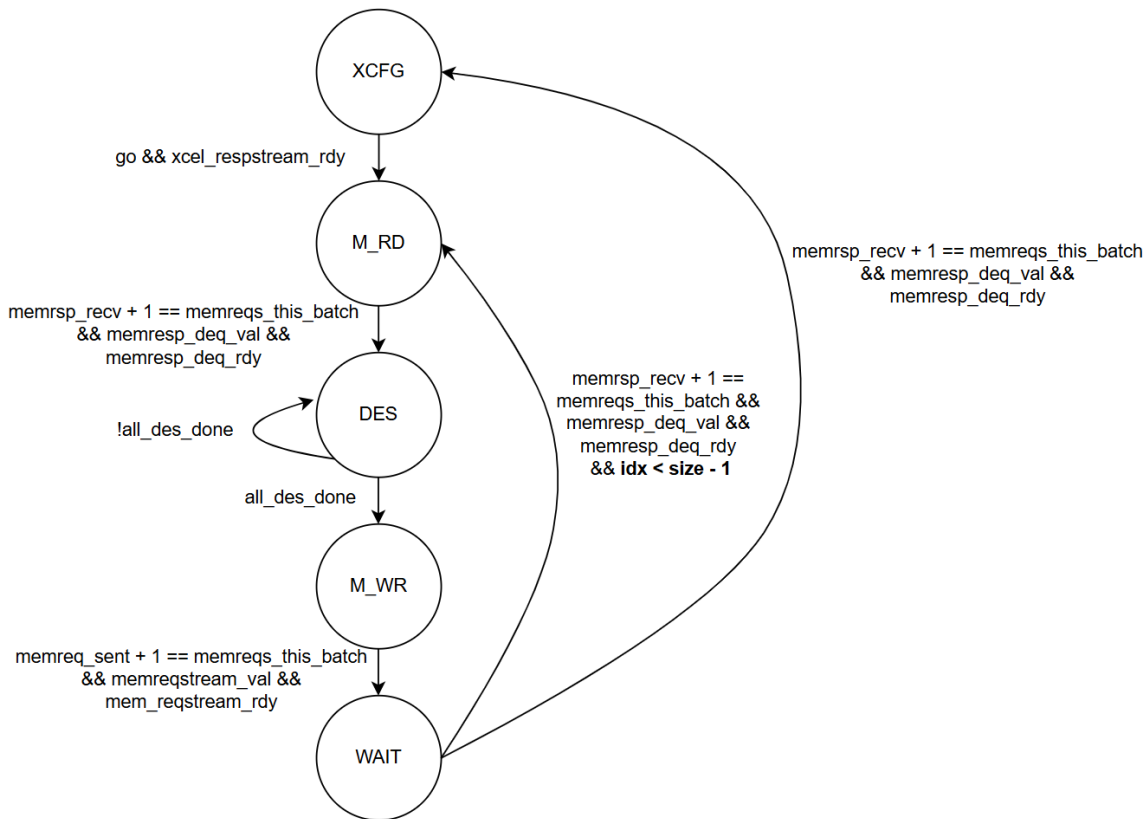


Figure 9. Parametrized DES FSM

Each response is unpacked and distributed across the pt_reg array using a core_idx which is derived from the current count of the memory responses received so far. The memory writes follow the same process. To manage the memory addresses across the cores, batching logic is used to process groups of up to NCORES blocks in a single batch. The batch_base determines the starting block index for each batch, and all memory accesses and core assignments are

computed via this base. The FSM iterates through batches using the idx counter. Batching enables efficient utilization of cores while handling edge cases at the end with partial final batches.

```
assign memreqs_this_batch = (batch_blocks_left + 1) >> 1;
assign batch_base = idx * NCORES;
```

4. Testing Strategy

Ensuring correctness is critical for cryptographic systems, as even small implementation errors can lead to incorrect results or security vulnerabilities. To validate the software and alternative designs, a comprehensive testing strategy was used to verify functional correctness across multiple inputs and operating conditions. The testing approach includes known-answer-tests (KATs), edge-case tests for bit flips, randomized testing, variable message block lengths, and varying modes of operation. Both ECB and CTR designs were individually tested since their algorithms produce different results.

4.1 Software Baseline Testing

The software baseline testing suite provides systematic verification of the correctness and robustness of the design to ensure both ECB and CTR behave correctly for comparison against the hardware accelerator. First, KATs for encryption and decryption are used to validate correctness against trusted outputs. The fixed plaintext-key pairs with known ciphertext results ensure the algorithm is implemented exactly as expected, and provide greater confidence before relying on test cases generated using the Python's PyCryptodome library. Next, randomized test vectors were used to test on arbitrary input data. These tests ensure the implementation isn't fitting to specific patterns and strengthen the robustness of the design. Then, the round-trip tests verify functional consistency across encryption and decryption.

In addition, specific ECB mode tests target multi-block behavior to verify the handling of different message lengths. These tests are important because ECB operates independently on each block, so iterating over blocks and memory indexing must be consistent across encryption and decryption. Finally, CTR mode tests validate the counter logic. These include tests for correct counter generation, with edge cases for nonce carry and overflow with full-wrap around of the counter. The self-inverse test again was used to confirm that encryption and decryption are identical in CTR. The multi-block tests also confirmed the intended operation of the counter across multiple blocks and longer messages. The software baseline testing was crucial to establish the expected behavior, including byte endianness of the output across multiple blocks, so that the accelerator emulates the same order.

4.2 Alternative Design Testing

Testing began with a bottom-up approach, starting with unit tests for individual submodules before moving on to the full system level validation. The S-box module was tested in isolation by driving five directed input vectors covering all zeros, all ones, and alternating bits as well as a high bit pattern to test the boundary conditions at the most significant bit. These vectors were also tested under source and sink delays to confirm behavior. The Feistel module was also tested in isolation using test vectors that cover all zeros and all one inputs for both the 32-bit R half as well as the 48-bit subkey. A single-bit input is used to verify that the E-box wraps around correctly back to the first position during the expansion. Another test using all zeros in the R input with an all ones subkey checks to see if the XOR path works correctly. Finally we had multiple S-box activation tests using a 6-bit group corresponding to each of the S-boxes to isolate and check each individual S-box.

Testing for the full system followed a blackbox approach throughout, verifying outputs against known answer tests (KATs). Multiple directed KATs were included to incorporate a large range of plaintext and key values. The encryption module previously written had directed tests with all zeros, all ones edge cases, and three alternating bit pattern edge cases. We also included randomized testing through the use of a test vector file, which was generated and validated against expected ciphertext values with an external module. For our decryption and CTR modes, these tests were reused to confirm that adding these modes would not regress any existing encryption behavior.

For the decryption, directed tests were added using the same data that the encryption tests were based on. The same values of plaintext, keys, and ciphertext were reused, with the cipher text as input and the original plaintext as the

expected output. Multiple KATs were included for decryption as well, mirroring those tested for encryption to confirm that the inverse operation holds for those specific vectors. Random vector tests were used similarly, where we used the same test vector file but processed the expected ciphertext as the input and the expected plaintext as the output. Round trip tests were also added to verify that encrypting then decrypting could correctly recover the original plaintext.

Our ECB mode required tests that spanned multiple blocks. Tests include block size coverage of 1, 2, 3 and 4 blocks with encryption and decryption at each size. The smaller block sizes, 1 and 2, verify if our boundary conditions at the minimum block count work as well as verify the independent processing of each adjacent block. The larger block tests consisting of 3, and 4 blocks, stress larger inputs, allowing for testing for correctness at a larger range. To verify for correctness across multiple cores, multi-core tests were added for 2 and 4 cores. The 2-core test has each core encrypt a different message with different keys, making sure that each core doesn't interfere with the other. The 4-core test adds on to the 2-core test by adding a mix of encryption and decryption across the four cores simultaneously. This helps test for cases that stress the cores across various workloads.

The CTR mode also required more tests in order to cover handling multiple blocks. A 1-block KAT was included to test our minimum-input boundary. Test cases also covered 2 block KAT, along with all zeros and all ones inputs, to make sure that those still worked from the single block encryption and decryption. Additional 2-block KATs using different keys were also tested to confirm that the output was key dependent. To test for larger inputs, 4, 5, 6 and 8 blocks were tested to stress the counter increment across successive blocks. Two corner cases were tested here, where one checked when the lower 32 bit word is set to 0xFFFFFFFF to verify the 64 bit counter rolls over correctly into the upper word and the other executed a full round while running the same CTR call on the ciphertext to confirm the original plaintext was recovered. Another nonce carry test was also included for CTR mode to check for counter rollover. This set the initial nonce to 0x00000000FFFFFFFF so the first counter increment would cause the lower half to overflow and carry into the upper half, allowing us to see if the 64-bit counter arithmetic was implemented correctly. Multi-core tests were also added for CTR mode, checking for 2 and 4 simultaneous cores. The 2-core test uses two of the same key, but different nonces for each core, to make sure the nonce state works independently between cores. The 4-core test assigns each core a different key, nonce and block count, with a nonce carry and nonce wraparound allowing for us to validate the correctness of the multicore.

ECB Encryption	ECB Decryption	CTR Mode
Directed KATs	Reversed ECB encrypt KATs	2-block KAT
All zeros/all ones	Ciphertext input, plaintext output	All zeros/all ones
Alternating bit patterns	Random vector tests	4 and 8 block stress tests
Random vector tests	Round trip: encrypt -> decrypt	Nonce carry overflow test
2 Cores	4 Cores	Round trip
		2 Cores
		4 Cores

Table 3. ECB and CTR Test Case Table

Line traces were used for debugging and stepping through the cycles of the accelerator to confirm expected behavior. The example below is for ECB decryption of 3 blocks using 2 cores. The traces show the initial processor writes to configure the source address and destination address in memory, the key, whether the mode is encrypt or decrypt, and the size of the message in blocks. Once the processor write go, the accelerator knows to begin reading from memory, starting from the given source address. The first batch of input text is read, and the start signal is set

to begin the DES cycles. The base is given in terms of the block index, so it begins at 0, whereas n refers to the number of cores active during this batch. After the computations across all cores are complete, the all_done flag is raised and the resulting output can be seen. The write request to memory can be seen, with the address and the length of the message. Once the accelerator receives the acknowledgement that the memory has been written, the next batch begins, since there is still 1 block to process. The FSM goes back to the read state to retrieve the next inputs from memory, using the incremented base index, which is now 2 blocks away. This time, n=1 since only one DES core is used. The written message to memory has an address that is 2 bytes off from the destination, since that was the length of the previous message. The length of this last message is only 8 bytes, because only 1 block is being written. Finally, when the write ack is received and the accelerator sees there are no more blocks to process, it sets the done_reg high which is read by the processor to signify completion of the entire message.

```

1r: > (CFG |
2r: > (CFG |
3: > (CFG |
4: > (CFG |
5: > (CFG |
6: > (CFG |
7: > (CFG |
8: > (CFG |
9: wr:01:00001000 > wr:01:00001000 (CFG |
10: . > . (CFG |
11: . > . (CFG |
12: wr:02:00002000 > wr:02:00002000 (CFG | src=00001000
13: . > . (CFG |
14: . > . (CFG |
15: # > # (CFG |
16: # > # (CFG |
17: # > # (CFG |
18: wr:03:faceface > wr:03:faceface (CFG | dst=00002000
19: . > . (CFG |
20: . > . (CFG |
21: # > # (CFG |
22: # > # (CFG |
23: # > # (CFG |
24: wr:04:faceface > wr:04:faceface (CFG | key_lo=faceface
25: . > . (CFG |
26: . > . (CFG |
27: # > # (CFG |
28: # > # (CFG |
29: # > # (CFG |
30: wr:05:00000001 > wr:05:00000001 (CFG | key_hi=faceface
31: . > . (CFG |
32: . > . (CFG |
33: # > # (CFG |
34: # > # (CFG |
35: # > # (CFG |
36: wr:06:00000003 > wr:06:00000003 (CFG | dec=1
37: . > . (CFG |
38: . > . (CFG |
39: # > # (CFG |
40: # > # (CFG |
41: # > # (CFG |
42: wr:00:00000001 > wr:00:00000001 (CFG | nblk= 3
43: . > . (CFG |
44: . > . (CFG |
45: # > # (CFG |
46: # > # (CFG |
47: # > # (CFG |
48: rd:00: > rd:00: (CFG | wr_go
49: . > . (RD | batch=1/2 | rd_req addr=00001000
50: . > . (RD |
51: . > . (RD |
52: . > . (RD | pt[0]=0011223344556677 pt[1]=8899aabbccddeeff
53: . > . (DES | all_done=0 | des_start base=0 n=2
54: . > . (DES |

```

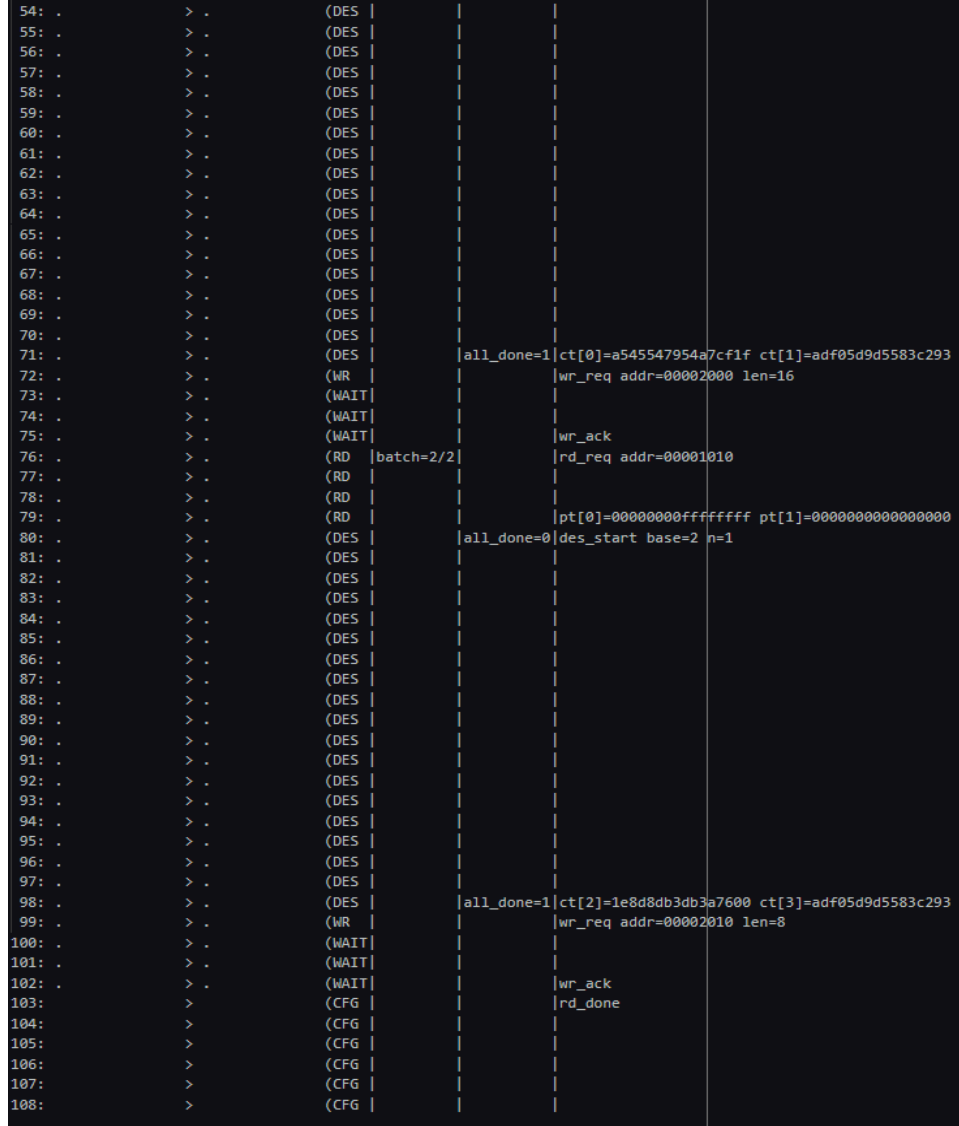


Figure 10. Line Traces for ECB Decryption of 3 Blocks Using 2 Cores

5. Evaluation

Initial evaluation was performed on the ECB and CTR accelerators in isolation. The accelerators were evaluated on a set of random data generated using Python libraries. The purpose of a larger data set size of 128 randomly generated inputs was to minimize the effects of setup overhead and test on more generalized workloads. Analysis results found that CTR performs similarly to or better than ECB, while providing stronger security, so we chose to continue further analysis with the CTR algorithm. Furthermore, the addition of parameterization allowed for design space exploration of the number of cores to use in the accelerator. Finally, we exchanged evaluation results with the ASCON group to perform a comparison between our designs.

5.1 ECB vs CTR Comparison

	Xcel ECB Encryption	Xcel ECB Decryption	Xcel CTR
Area (μm^2)	159628.358		133854.515

num_cycles	815	815	816
Power (mW)	18.0	18.0	12.7
Energy (nJ)	146.7	146.7	103.632

Table 4. DES ECB vs CTR Accelerator Evaluation Data

From a performance perspective, the cycle counts are extremely similar between the two block cipher modes. When comparing encryption on the software baselines, CTR mode only uses 958 more cycles out of the 265,063 cycles for ECB, which is a 0.36% increase. The alternative designs show that CTR mode has a few more cycles compared to ECB mode. This makes sense because CTR mode only adds additional counter logic and an XOR operation; the state machines function similarly across the two designs. CTR mode has lower power, energy consumption, and area compared to ECB, due to the encryption DES core, which uses only one subkey gen module rather than instantiating 16 to support decryption functionality.



Figure 11. ECB vs CTR Evaluation Data Graphs

The most important tradeoff between the two cipher block modes is the qualitative difference in security. CTR is significantly stronger because every block has its own unique counter value, producing different DES core outputs given the same key across multiple blocks, which removes ECB's pattern leakage due to identical plaintext to ciphertext blocks. In addition, both designs are highly parallelizable, but CTR does not require a separate decryption path, while ECB requires the datapath to be modified to support decryption. Thus, CTR provides higher security encryption that supports parallel processing, which in turn enables high-speed encryption suitable for cloud platforms, VPNs, and TLS.

5.2 CTR Baseline vs Alternative Results

	Proc CTR	Proc + Xcel CTR
Area (μm^2)	195925.990	328551.194

num_cycles	266021	832
CPI	1.01	29.71
Power (mW)	22.2	18.4
Energy (nJ)	59056.6664	153.0917

Table 5. DES CTR Baseline vs Alternative Evaluation Data

The results for the DES CTR baseline software implementation and alternative accelerator implementation show a significant advantage for the alternative design. In terms of cycle count, the software baseline took 266,021 cycles, while the processor + accelerator design completes in 832 cycles, which corresponds to roughly a 320x speedup. The 16 additional cycles from the composition with the processor compared to the accelerator in isolation are due to processor control overhead, where the processor must issue configuration and control messages to the accelerator. The large gap in performance is primarily due to the sequential implementation of the processing each block in software, while the hardware is combinational and exploits parallelism, so that it can process multiple blocks simultaneously. Because DES was designed for hardware with most of the datapath being combinational, the run time is significantly reduced.

In terms of area, the processor baseline occupies $195,925.990 \mu\text{m}^2$, while the combined processor + accelerator has a total area of $328,551.194 \mu\text{m}^2$, which is roughly a 68% increase. From the area reports, the largest individual unit contribution to the area of the accelerator comes from the DES core, which makes up roughly 36% of its total area, with additional area coming from the smaller submodules such as Feistel and Sbox. The majority of the rest is used for non-combinational logic, which reveals that registers use as much area as the DES combinational logic. This is expected as this accelerator contains many registers and queues for interfacing with the processor and memory. The software baseline runs on the TinyRV2 processor, which is more flexible than the accelerator, but performs poorly for this compute-heavy application. The CPI of 1.01 shows that the processor itself is running efficiently, but the DES algorithm requires too many instructions and cycles when implemented on the baseline software. This makes the additional area tradeoff of the accelerator worth it, as it can provide higher throughput. On the other hand, the high CPI of the accelerator alternate design is due to the processor stalling while waiting for the accelerator to complete the computations. The waiting cycles are counted even though the work is offloaded to the accelerator.

The energy usage also improved substantially with the alternative design. The baseline uses 59,056.6664 nJ while the alternative uses only 153.0917 nJ, which is about a 3x reduction. The alternative design also uses less power, with 18.4 mW consumed compared to 22.2 mW for the baseline, largely due to the reduction in execution time. Completing the workload faster leads to lower total energy consumption, so though there was less of a difference in power, the accelerator was much more energy-efficient. Looking at the detailed power report for the accelerator shows that 76.8% of the power is used by the DES core to run the Feistel and Sbox functions. The remaining power usage can be attributed to various queues, clocking gates, and control logic. Most of the power, aside from internal power, is switching power, meaning it comes from the dynamic activity of the small logic and clocked components rather than from leakage.

For other parameters, such as the number of DES cores used in the accelerator design, the accelerator would become more efficient as more blocks could be processed in parallel. The message length would also affect the efficacy of the baseline and alternative designs. The processor + accelerator design benefits the most with longer messages, as the setup and communication overhead becomes less significant compared to short messages.

5.3. Parametrized Cores

Area vs Time for Parameterized ECB & CTR Cores

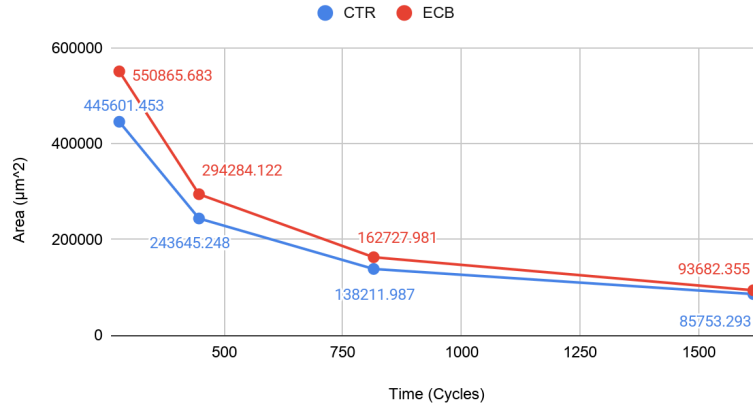


Figure 12. Area vs Execution Time for ECB and CTR Cores with NCORES = 1, 2, 4, & 8

Area vs Time for Parameterized ECB & CTR Cores

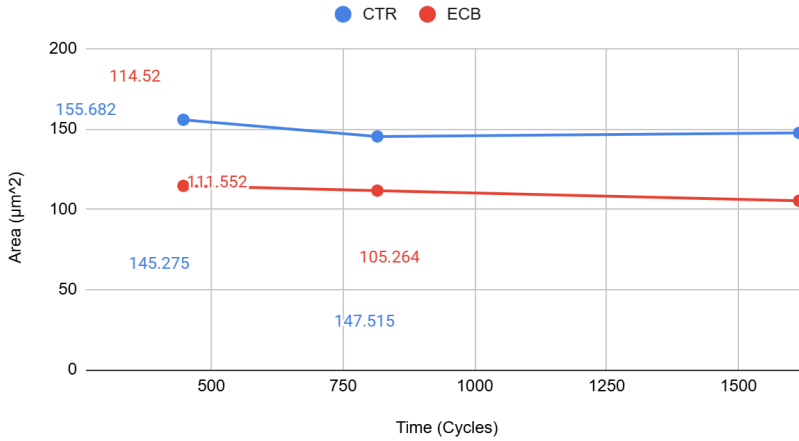
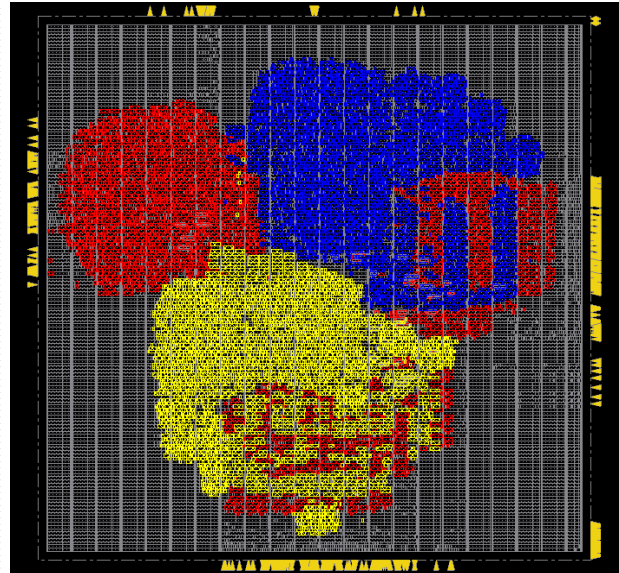
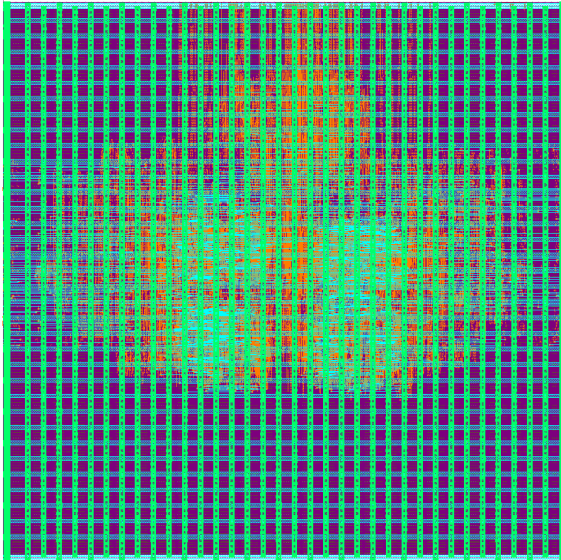


Figure 13. Energy vs Execution Time for ECB and CTR Cores with NCORES = 1, 2, 4, & 8

Both ECB and CTR accelerators were parametrized and evaluated at NCORES = 1, 2, 4, and 8. From the results, the area scales exponentially with the number of cores, with increasing area per core added due to factors such as additional control logic and increased routing overhead. The main cost of adding more cores for increased parallelism is the additional area. However, this area tradeoff allows for higher throughput, with total cycle time decreasing with additional cores. The largest speedup appears when going from 1 to 2 cores. This gives a roughly 2x speedup, while subsequent additions lead to diminishing returns (Figure 11). The fixed overhead adds logic required to support batching to even more cores, slowing cycle time. In addition, the memory bandwidth starts to be the main bottleneck, since more cores require more data to be read in per batch. The energy sees only a slight linear increase as the number of cores becomes large. In fact, the energy first decreases from 1 to 2 cores, despite more cores using higher power, due to the drastic reduction in cycle time. But as NCORES grows, the area grows faster than the reduction in cycle time, leading to gradually increasing energy consumption. Overall, increasing the number of DES cores improves performance substantially, but at the cost of rapidly increasing area with eventual diminishing energy efficiency.

ECB Mode

Accelerator Block Level Results



```
timestamp           = 2026-05-01 16:19:22
design_name           = Proj3Xcel_ECB_noparam
clock_period         = 10.0
pymtl-rtlsim        = 5/5 passed
vcs-rtlsim          = 14/14 passed
synth_setup_slack    = 2.9943 ns
synth_num_stdcells   = 6881
synth_area           = 122345.081 um^2
run (synthesis)      = no errors
ffglsim             = 14/14 passed
pnr_setup_slack      = 2.1069 ns
pnr_hold_slack       = 0.0101 ns
pnr_clk_ins_src_lat  = 0 ns
pnr_density          = 27.12%
pnr_num_stdcells     = 9734
pnr_cell_area        = 159628.358 um^2
pnr_core_area        = 612019.565 um^2
run (pnr)            = no errors
sta_setup_slack      = 2.1253 ns
sta_hold_slack       = 0.0022 ns
baglsim             = 14/14 passed
main-drc             = no violations found
antenna-drc          = no violations found
lvs                 = no violations found
```

```
Proj3Xcel_ECB_noparam_proj3-xcel-ecb-sim-rtl-encrypt
- exec_time = 40 cycles
- exec_time = 400.0000 ns
- power     = 13.2000 mW
- energy    = 5.2800 nJ

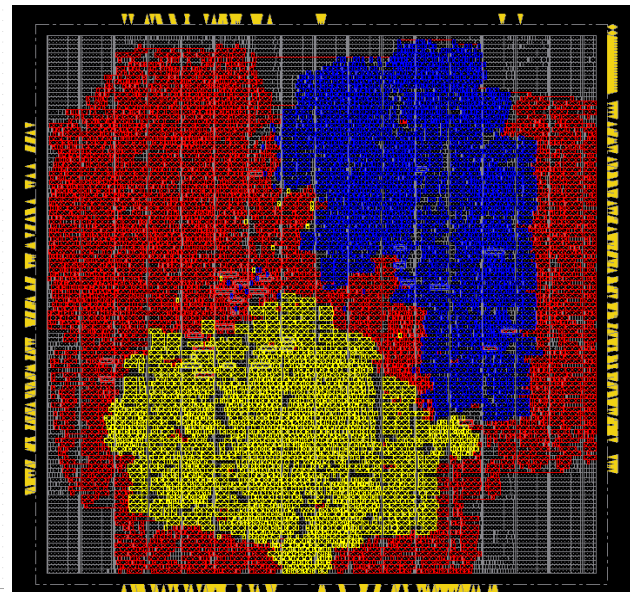
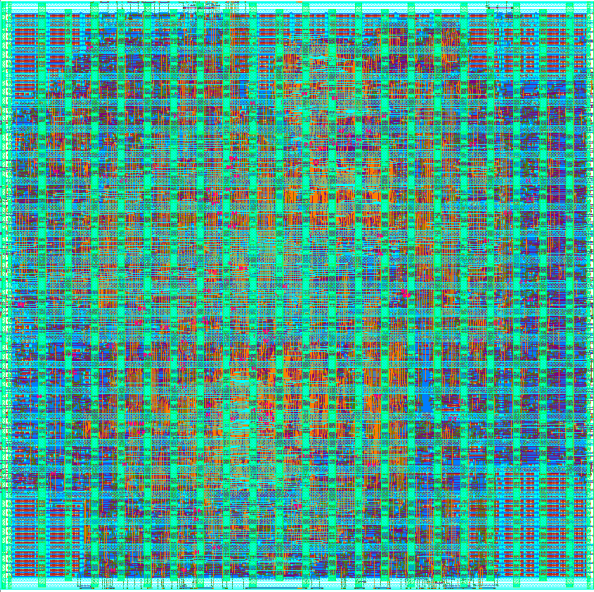
Proj3Xcel_ECB_noparam_proj3-xcel-ecb-sim-rtl-decrypt
- exec_time = 40 cycles
- exec_time = 400.0000 ns
- power     = 13.2000 mW
- energy    = 5.2800 nJ

Proj3Xcel_ECB_noparam_proj3-xcel-ecb-sim-rtl-random-encrypt
- exec_time = 815 cycles
- exec_time = 8150.0000 ns
- power     = 18.0000 mW
- energy    = 146.7000 nJ

Proj3Xcel_ECB_noparam_proj3-xcel-ecb-sim-rtl-random-decrypt
- exec_time = 815 cycles
- exec_time = 8150.0000 ns
- power     = 18.0000 mW
- energy    = 146.7000 nJ
```

CTR Mode

Accelerator Block Level Results



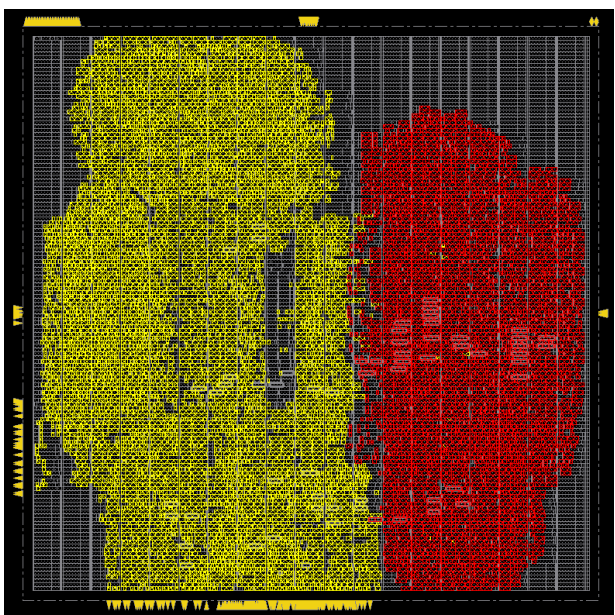
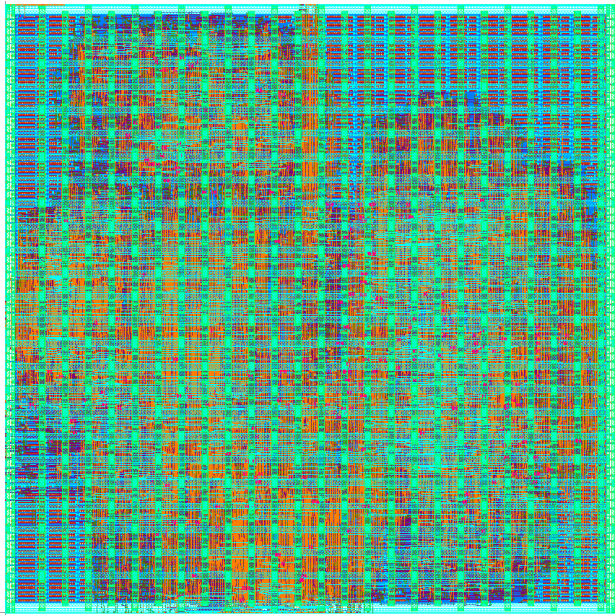
```
timestamp           = 2026-05-01 17:38:19
design_name           = Proj3Xcel_CTR_noparam
clock_period         = 10.0
pymtl-rtlsim         = 3/3 passed
vcs-rtlsim           = 12/12 passed
synth_setup_slack    = 0.5538 ns
synth_num_stdcells   = 6207
synth_area           = 117434.418 um^2
run (synthesis)      = no errors
ffglsim              = 12/12 passed
pnr_setup_slack      = 0.1157 ns
pnr_hold_slack       = 0.0173 ns
pnr_clk_ins_src_lat  = 0 ns
pnr_density          = 64.71%
pnr_num_stdcells     = 8378
pnr_cell_area        = 146077.389 um^2
pnr_core_area        = 234908.352 um^2
run (pnr)            = no errors
sta_setup_slack      = 0.1596 ns
sta_hold_slack       = 0.0131 ns
baglsim              = 12/12 passed
main-drc              = no violations found
antenna-drc          = no violations found
lvs                  = no violations found
```

```
Proj3Xcel_CTR_noparam_proj3-xcel-ctr-sim-rtl-basic
- exec_time = 41 cycles
- exec_time = 410.0000 ns
- power     = 11.6000 mW
- energy    = 4.7560 nJ
```

```
Proj3Xcel_CTR_noparam_proj3-xcel-ctr-sim-rtl-random
- exec_time = 816 cycles
- exec_time = 8160.0000 ns
- power     = 15.3000 mW
- energy    = 124.8480 nJ
```

All checks passed

Processor Baseline Design Results

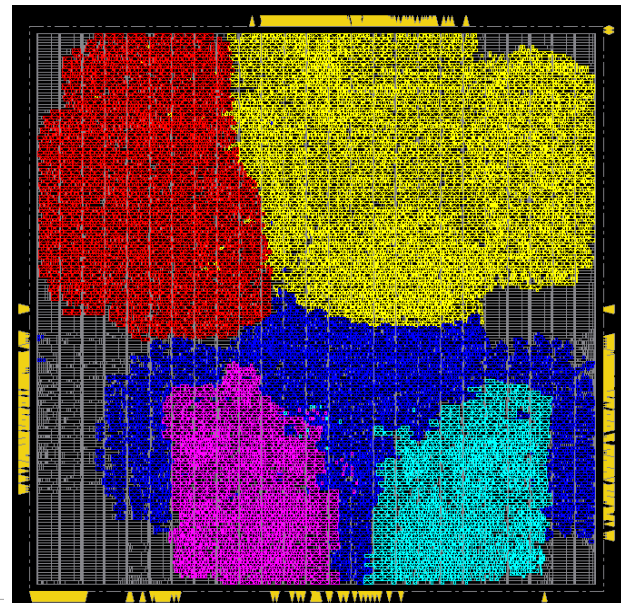
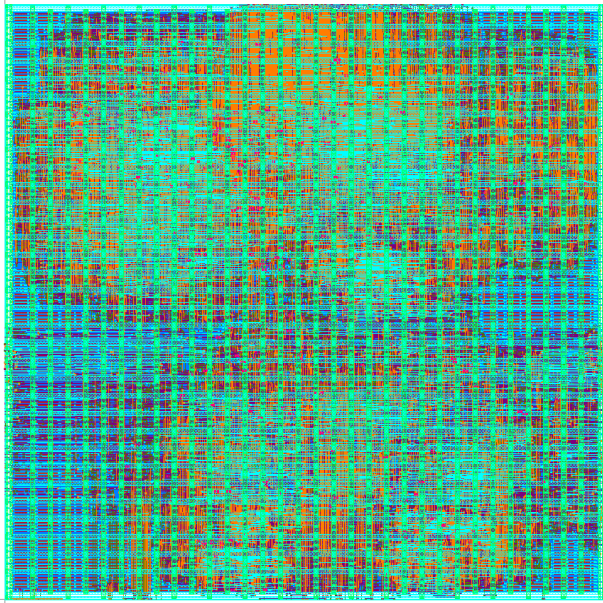


```
timestamp           = 2026-05-01 20:05:35
design_name          = ProcMemXcel_v0_NullXcel
clock_period        = 10
pymtl-rtlsim        = 0/0 passed
vcs-rtlsim          = 2/2 passed
synth_setup_slack   = 0.0275 ns
synth_num_stdcells  = 7941
synth_area          = 160012.519 um^2
run (synthesis)     = no errors
ffglsim             = 2/2 passed
pnr_setup_slack     = 0.0832 ns
pnr_hold_slack      = 0.0162 ns
pnr_clk_ins_src_lat = 0 ns
pnr_density         = 63.70%
pnr_num_stdcells    = 10150
pnr_cell_area       = 195925.990 um^2
pnr_core_area       = 320218.214 um^2
run (pnr)           = no errors
sta_setup_slack     = 0.1202 ns
sta_hold_slack      = 0.0123 ns
baglsim             = 2/2 passed
main-drc            = no violations found
antenna-drc         = no violations found
lvs                 = no violations found
```

```
ProcMemXcel_v0_NullXcel_pmx-sim-null-rtl-proj3-baseline-ctr-eval
- exec_time = 266021 cycles
- exec_time = 2660210.1970 ns
- power     = 22.2000 mW
- energy    = 59056.6664 nJ

All checks passed
```

Processor + Accelerator Alternative Design Results



```
timestamp                = 2026-05-01 21:35:32
design_name                = ProcMemXcel_v2_Proj3Xcel_CTR
clock_period              = 10
pyrtl-rtlsim              = 0/0 passed
vcs-rtlsim                = 2/2 passed
synth_setup_slack         = 0.0188 ns
synth_num_stdcells        = 14009
synth_area                = 272779.942 um^2
run (synthesis)           = no errors
ffglsim                   = 2/2 passed
pnr_setup_slack           = 0.0559 ns
pnr_hold_slack            = 0.0113 ns
pnr_clk_ins_src_lat       = 0 ns
pnr_density                = 65.02%
pnr_num_stdcells          = 18650
pnr_cell_area             = 340811.386 um^2
pnr_core_area             = 545586.227 um^2
run (pnr)                 = no errors
sta_setup_slack           = 0.0887 ns
sta_hold_slack            = 0.0081 ns
baglsim                   = 2/2 passed
main-drc                  = no violations found
antenna-drc               = no violations found
lvs                        = no violations found
```

```
ProcMemXcel_v2_Proj3Xcel_CTR_pmx-sim-proj3-ctr-rtl-proj3-xcel-ctr-eval
- exec_time = 832 cycles
- exec_time = 8320.1950 ns
- power     = 21.1000 mW
- energy    = 175.5561 nJ
```

All checks passed

5.4 ASCON Comparison

We exchanged our results with the ASCON Lightweight Cryptography Accelerator group. The ASCON-128 algorithm is a sponge-based authenticated encryption algorithm, so it operates on states rather than independent blocks. Fundamentally, the cryptography algorithms of DES and ASCON are different, with ASCON processing data serially through a shared state while DES processes in parallel per block. Each algorithm has its own tradeoffs between area, cycle time, energy efficiency, and security.

Input message: "ECE 6745"

Ascon-128 Accelerator with Lightweight Cryptography

```
timestamp           = 2026-05-12 17:37:13
design_name          = ProcMemXcel_v2_Proj4Xcel
clock_period        = 10
pymtl-rtlsim        = 0/0 passed
vcs-rtlsim          = 1/1 passed
synth_setup_slack   = 0.0155 ns
synth_num_stdcells  = 20762
synth_area          = 394934.039 um^2
run (synthesis)     = no errors
ffglsim             = 1/1 passed
pnr_setup_slack     = 0.1120 ns
pnr_hold_slack      = 0.0139 ns
pnr_clk_ins_src_lat = 0 ns
pnr_density         = 67.93%
pnr_num_stdcells    = 29096
pnr_cell_area       = 516010.298 um^2
pnr_core_area       = 790311.514 um^2
run (pnr)           = no errors
sta_setup_slack     = 0.1364 ns
sta_hold_slack      = 0.0078 ns
baglsim            = 1/1 passed
main-drc            = no violations found
antenna-drc         = no violations found
lvs                = no violations found
```

ProcMemXcel_v2_Proj4Xcel_pmx-sim-proj4-rtl-ECE6745

```
exec_time = 113 cycles
exec_time = 1130.2130 ns
power     = 25.0000 mW
energy    = 28.2553 nJ
```

DES CTR 1 Core

```
timestamp           = 2026-05-10 17:36:50
design_name          = Proj3Xcel1_CTR_v2__ncores_1
clock_period        = 10.0
pymtl-rtlsim        = 4/4 passed
vcs-rtlsim          = 13/13 passed
synth_setup_slack   = 1.4987 ns
synth_num_stdcells  = 3521
synth_area          = 72871.859 um^2
run (synthesis)     = no errors
ffglsim             = 13/13 passed
pnr_setup_slack     = 0.5118 ns
pnr_hold_slack      = 0.0106 ns
pnr_clk_ins_src_lat = 0 ns
pnr_density         = 61.29%
pnr_num_stdcells    = 4537
pnr_cell_area       = 85753.293 um^2
pnr_core_area       = 145860.064 um^2
run (pnr)           = no errors
sta_setup_slack     = 0.5560 ns
sta_hold_slack      = 0.0090 ns
baglsim            = 13/13 passed
main-drc            = no violations found
antenna-drc         = no violations found
lvs                = no violations found
```

```
Proj3Xcel1_CTR_v2__ncores_1_proj3-xcel-ctr-v2-sim-rtl-xcomparison
- exec_time = 41 cycles
- exec_time = 410.0000 ns
- power     = 6.3050 mW
- energy    = 2.5850 nJ
```

The ASCON accelerator is significantly larger, with a post pnr area of approximately 516,010 μm^2 compared to 85,753 μm^2 for DES, which is around a 6x increase. This reflects ASCON's larger internal state of 320 bits and 128-bit key, while DES uses smaller 64-bit blocks with simpler round logic. In terms of performance, DES executes in 41 cycles, while ASCON requires 113 cycles. This difference comes from the difference in encryption algorithms, where ASCON must sequentially apply a transformation across its internal states, while DES consists of a relatively shorter round pipeline. The energy difference of ASCON compared to DES is also around 10x higher, largely due to the higher area and longer execution time. Overall, DES CTR is much more efficient for this smaller message and extremely parallelizable, however ASCON exchanges higher area and area overhead for stronger modern cryptographic properties providing authenticated encryption and higher security.

References

- [1] Texas Instruments, "Hardware Cryptographic Acceleration on Sitara™ Processors," Application Report SPRY198A, 2016. <https://www.ti.com/lit/wp/spry198a/spry198a.pdf>
- [2] GeeksforGeeks, "Block Cipher modes of Operation," *GeeksforGeeks*, Jul. 05, 2018. <https://www.geeksforgeeks.org/ethical-hacking/block-cipher-modes-of-operation/>
- [3] A. Rouvroy, F.-X. Standaert, J.-J. Quisquater, and J.-D. Legat, "Efficient Uses of FPGAs for Implementations of DES and Its Experimental Linear Cryptanalysis," *IEEE Transactions on Computers*, vol. 52, no. 4, pp. 473–482, Apr. 2003.
- [4] A. Satoh, S. Morioka, K. Takano, and S. Munetoh, "A Compact Rijndael Hardware Architecture with S-Box Optimization," in *Proc. ASIACRYPT*, 2001.
- [5] M. Chiosa, T. Benz, and G. Alonso, "Hardware Acceleration of Compression and Encryption in SAP HANA," *Proc. VLDB Endowment*, vol. 15, no. 12, pp. 3277–3290, Aug. 2022.
- [6] Rambus Inc., "3DES IP-16 Crypto Accelerator Core," 2026. <https://www.rambus.com/security/crypto-accelerator-cores/3des-ip-16/>
- [7] Arif Sasongko, I. M. Narendra Kumara, Arief Wicaksana, Frédéric Rousseau, and Olivier Muller. 2021. Hardware Context Switch-based Cryptographic Accelerator for Handling Multiple Streams. *ACM Trans. Reconfigurable Technol. Syst.* 14, 3, Article 14 (August 2021), 25 pages, DOI: <https://doi.org/10.1145/3460941>.
- [8] C. Batten, "ECE 6745 Tutorial 9: TinyRV2 Accelerator RTL Design," *ECE 6745 Complex Digital ASIC Design*, 2024. <https://cornell-ece6745.github.io/ece6745-2025s-mkdocs/ece6745-tut09-xcel-rtl/#1-baseline-tinyrv2-processor-fl-and-rtl-models>
- [9] C. Batten "ECE 6745 Lab 5: TinyRV2 Accelerators," *ECE 6745 Complex Digital ASIC Design*, 2024. <https://cornell-ece6745.github.io/ece6745-mkdocs/ece6745-lab5/>
- [10] GeeksforGeeks, "Data Encryption Standard (DES) | Set 1," *GeeksforGeeks*, Aug. 17, 2018. <https://www.geeksforgeeks.org/computer-networks/data-encryption-standard-des-set-1/>
- [11] M. Davio, Y. Desmedt, J. Goubert, F. Hoornaert, and Jean-Jacques Quisquater, "Efficient Hardware and Software Implementations for the DES.," *Advances in Cryptology, Proceedings of CRYPTO '84*, Santa Barbara, California, USA, August 19-22, 1984, Proceedings, pp. 144–146, Jan. 1984, doi: https://doi.org/10.1007/3-540-39568-7_14.
- [12] M. Dworkin, "Recommendation for Block Cipher Modes of Operation Methods and Techniques," 2001. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf>